

Computer Vision

Summer semester 2025

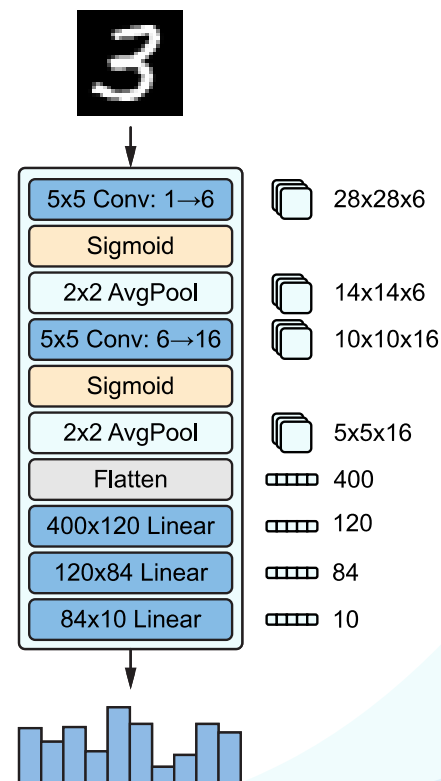
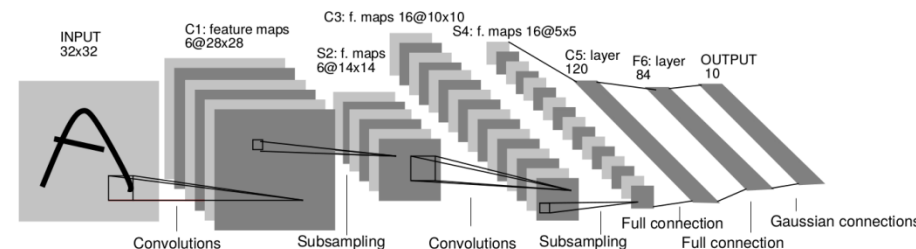
Lecture 10 – Convolutional Neural Networks III

22.05.2025

Prof. Bastian Leibe

Course Outline

- Image Processing Basics
- Segmentation & Grouping
- Object Recognition & Categorization
- **Deep Learning for Vision**
 - Convolutional Neural Networks (CNNs)
 - Deep Learning Background
 - **CNN Architectures**
 - CNNs for Object Detection
 - CNNs for Semantic Segmentation
 - CNNs for Matching
 - Transformers
- 3D Reconstruction



Announcement

- Keynote by Lucas Beyer (OpenAI)
 - One of the developers of the Vision Transformer (ViT)
 - (...and former PhD student of our group)



TAG DER INFORMATIK

DAY OF COMPUTER SCIENCE 2025



One Date, Three Locations: 5 June 2025

\$ JOB_FAIR.JSON < {
 "Time": "10:30 - 16:00",
 "Location": "Computer Science Center",
 "_info": "Get ice cream and party wristbands!"
}

\$ GRADUATION_CEREMONY.JSON < {
 "Time": "12:00 - 16:00",
 "Location": "Computer Science Center (Aula)",
 "_keynotes": "Dr. Lucas Beyer (OpenAI)
and Prof. Daniel Neider (TU Dortmund)"
}

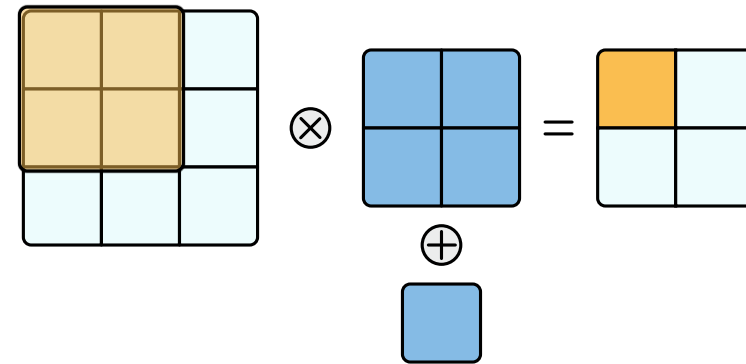
\$ PARTY.JSON < {
 "Time": "20:00 - 00:30",
 "Location": "Das LIEBIG, Liebigstr. 19",
 "_info": "Wristband required! Free drinks!"
}

MORE INFO AT WWW.TDI.AC



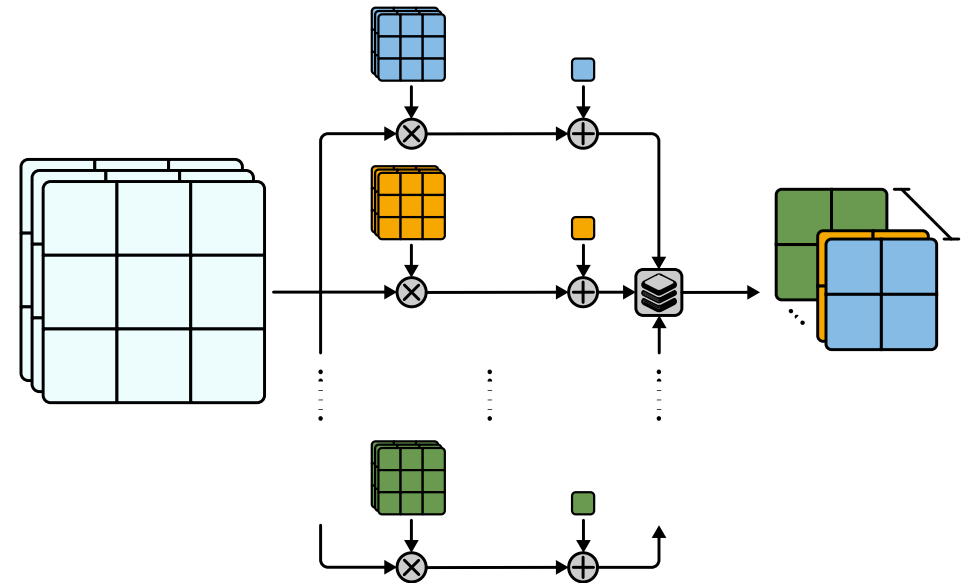
Convolutional Neural Networks

1. Motivation
2. **Learnable Filters**
3. Convolutional Layers
4. Other CNN Layers
5. CNN Architectures
6. Encoder-Decoder Structures



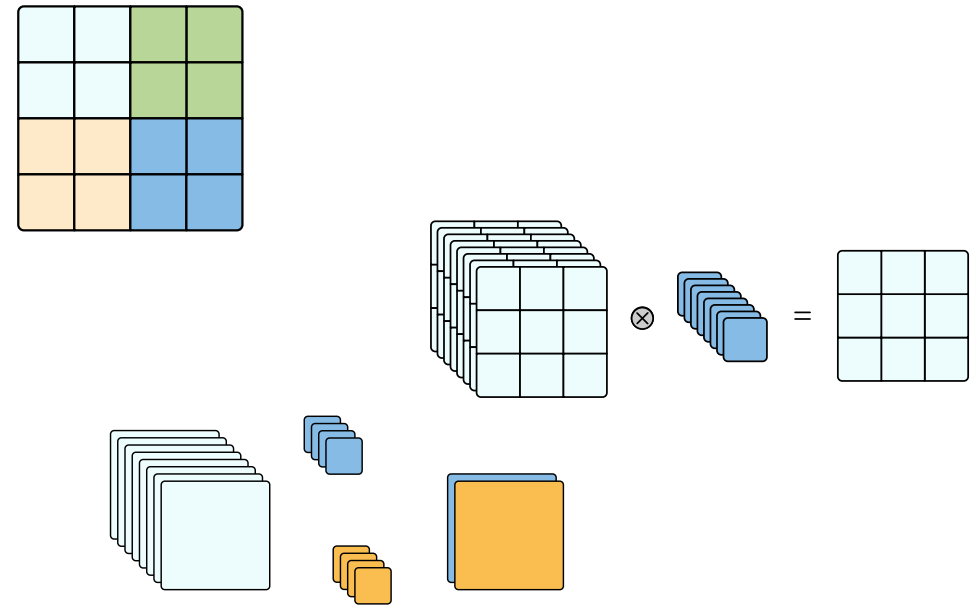
Convolutional Neural Networks

1. Motivation
2. Learnable Filters
- 3. Convolutional Layers**
4. Other CNN Layers
5. CNN Architectures
6. Encoder-Decoder Structures



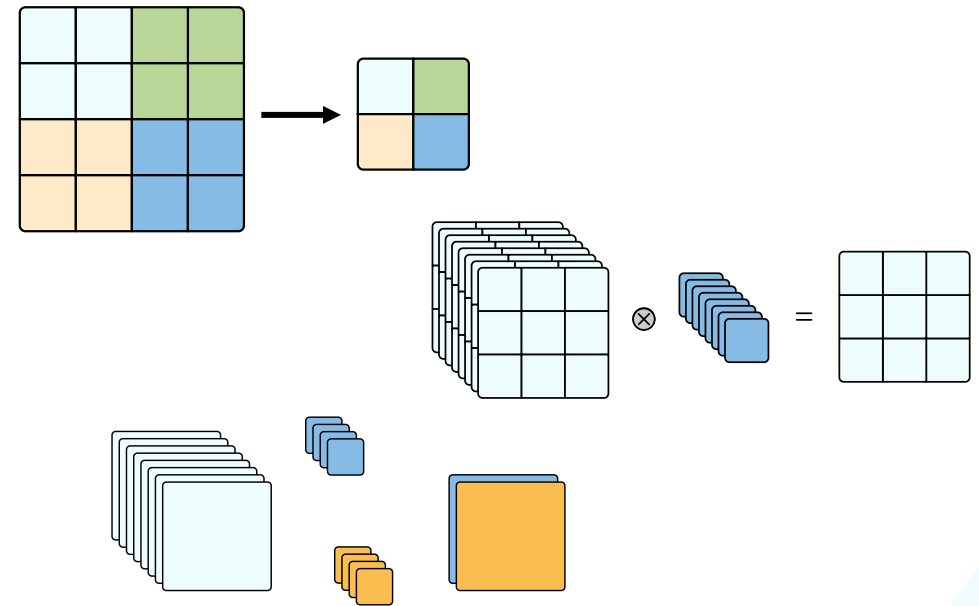
Convolutional Neural Networks

1. Motivation
2. Learnable Filters
3. Convolutional Layers
4. **Other CNN Layers**
5. CNN Architectures
6. Encoder-Decoder Structures



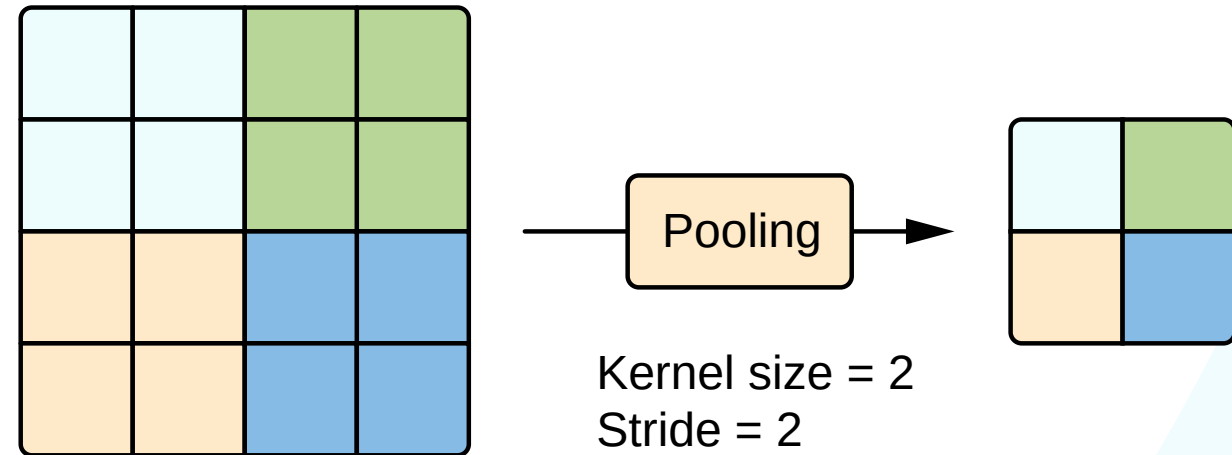
Other CNN Layers

- We have already introduced Convolutional layers.
 - We can apply activation functions and normalization layers as before.
- Other common building blocks:
 - **Pooling layers** to reduce spatial dimension and increase translational robustness.
 - **1x1 convolutions** to change the number of channels.
 - **Grouped convolutions** to further reduce the number of parameters.



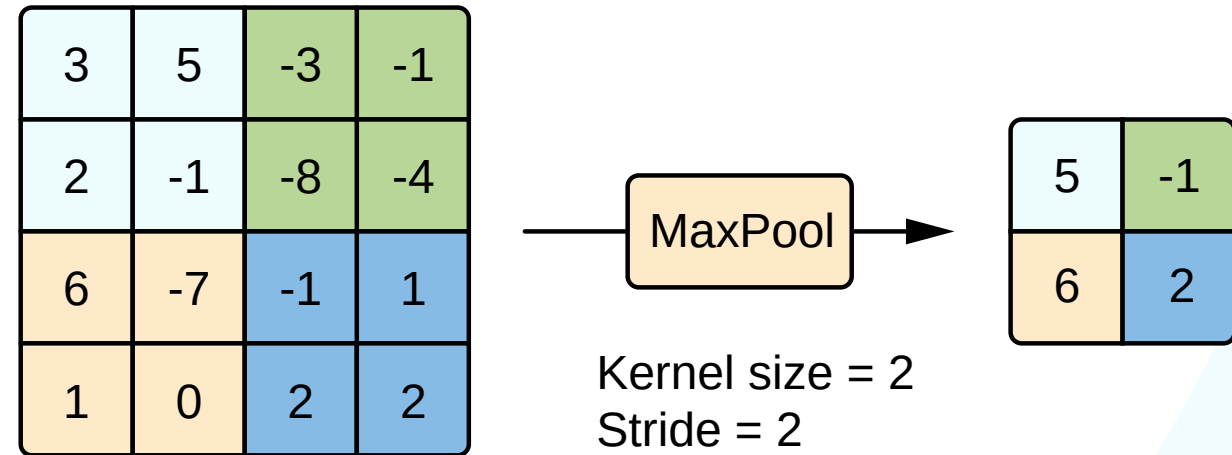
Pooling Layers

- Pooling applies an aggregation function to a local window.
 - This reduces the spatial size without additional parameters.
 - It also increases translational robustness.



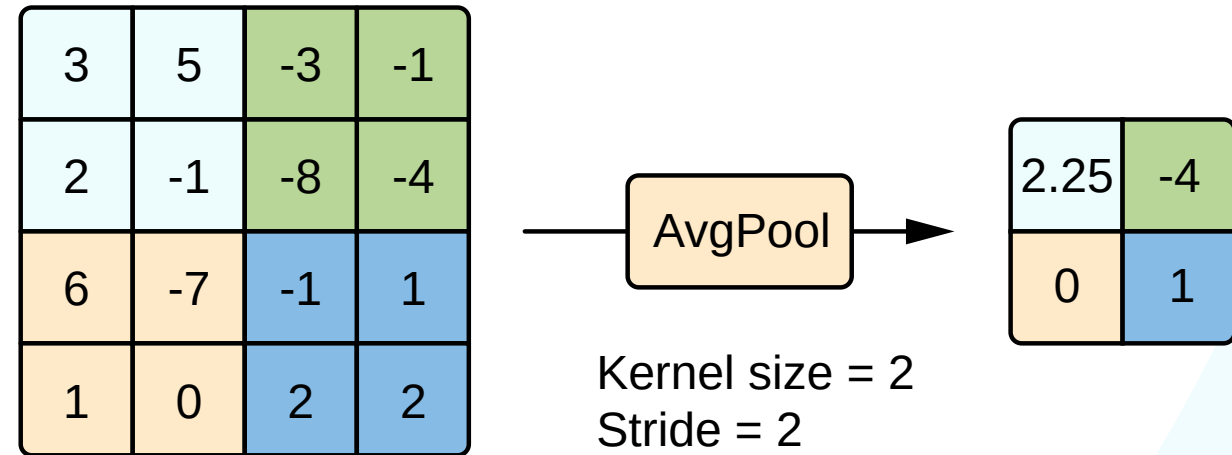
Pooling Layers

- Pooling applies an aggregation function to a local window.
 - This reduces the spatial size without additional parameters.
 - It also increases translational robustness.
- Popular choices:
 - Max-Pooling



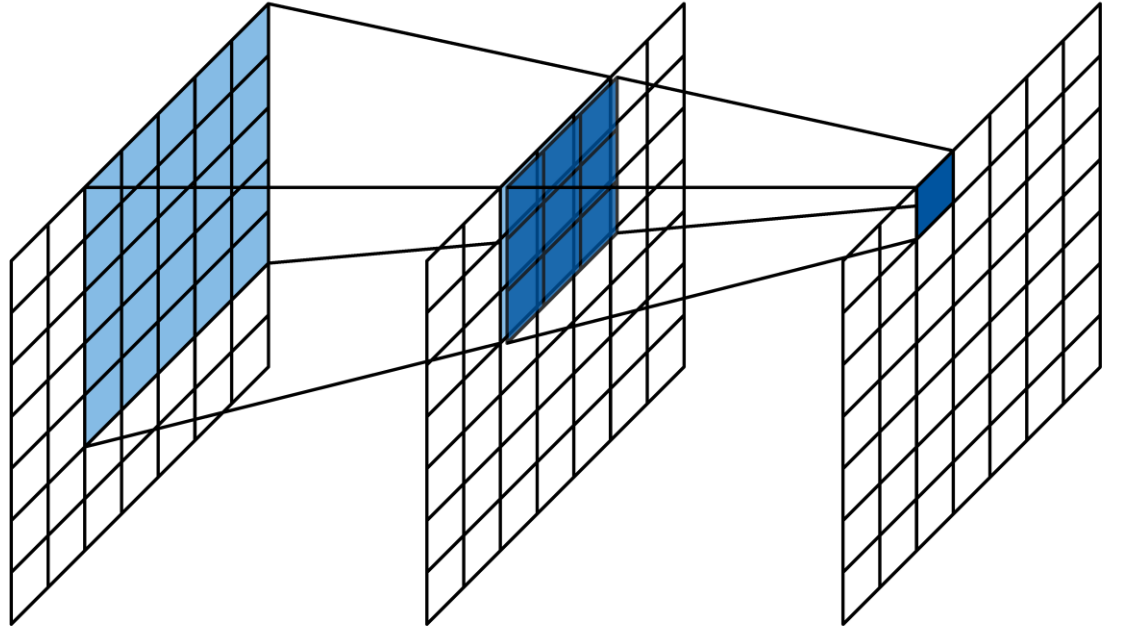
Pooling Layers

- Pooling applies an aggregation function to a local window.
 - This reduces the spatial size without additional parameters.
 - It also increases translational robustness.
- Popular choices:
 - Max-Pooling
 - Avg-Pooling
- Note:
 - We apply pooling to each channel separately



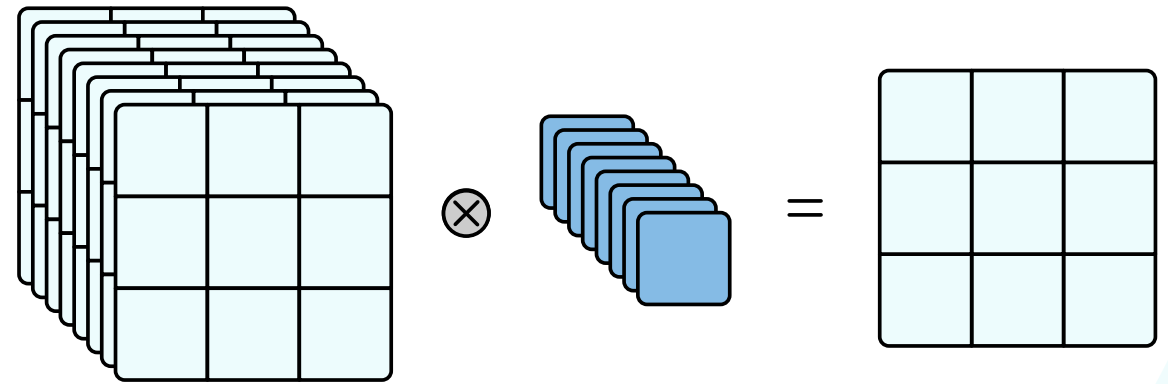
Receptive Field

- The **receptive field** is the subset of the input image or feature map that influences a specific neuron.
- Stacking two 3x3 filters (with stride 1) allows the neuron to see a 5x5 receptive field.
- Networks with small filters but multiple layers can still see the whole input image.

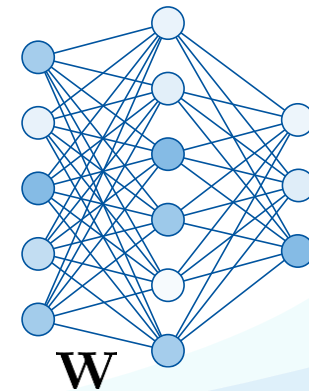


1x1 Convolutions

- Use a filter size of 1x1.
- Changes the number of channels without affecting spatial resolution, with small number of parameters.
- Useful tool in deeper networks.
 - Reduce channel count before costly operations.
 - Increase channel count for more expressivity.
- 1x1 convolutions are equivalent to fully-connected layers

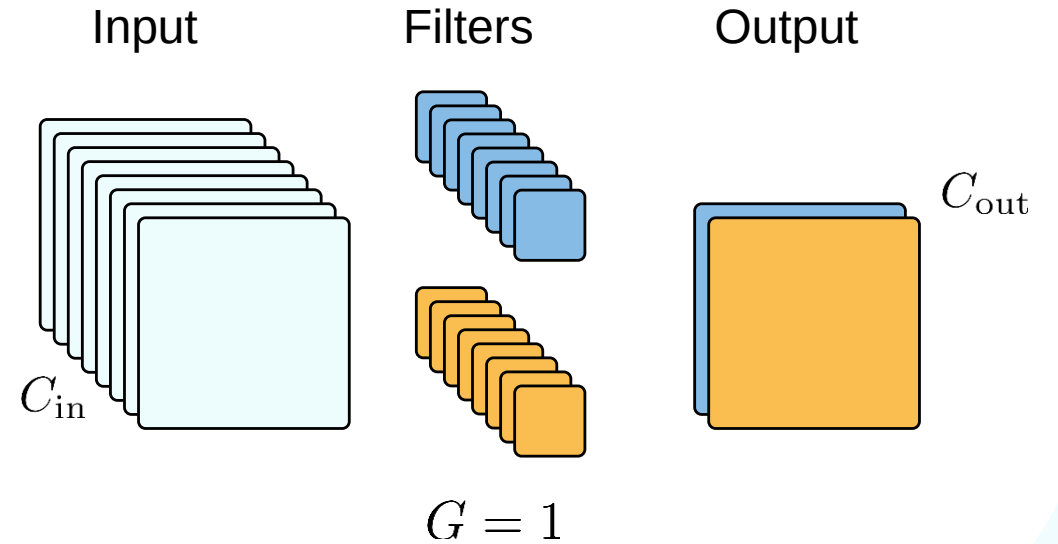


$$\#Params = C_{in} \cdot C_{out} \cdot 1 \cdot 1$$



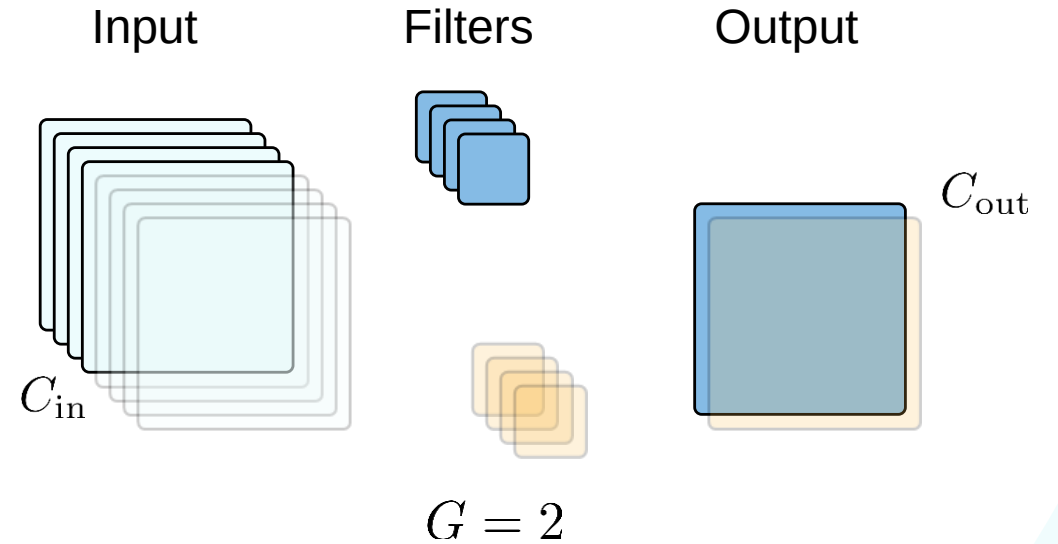
Grouped Convolution

- Divide input and output channels into G groups.
- Apply a separate convolution on each group and stack the results.
 - Reduces number of parameters by factor G .
 - Both C_{in} and C_{out} have to be multiples of G .



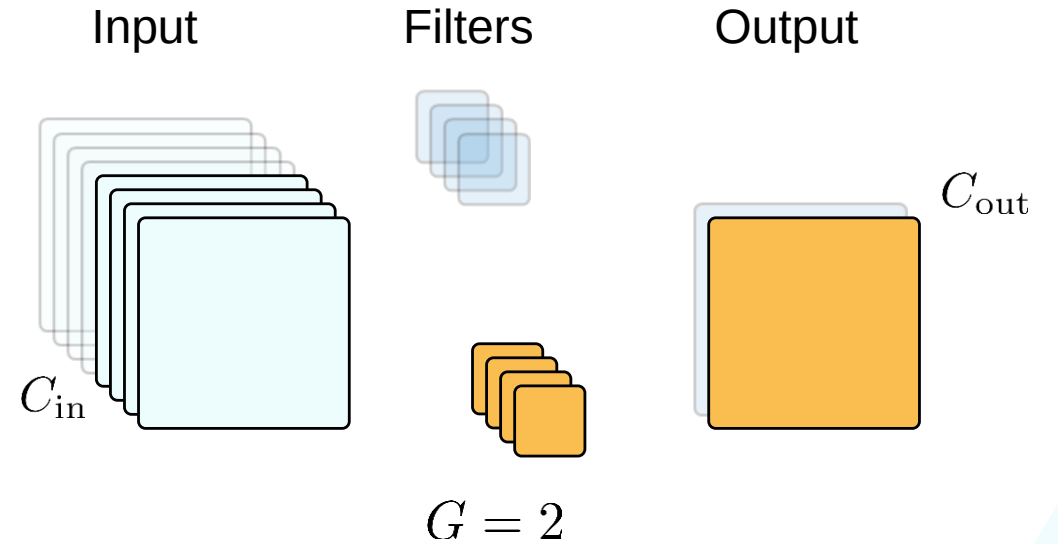
Grouped Convolution

- Divide input and output channels into G groups.
- Apply a separate convolution on each group and stack the results.
 - Reduces number of parameters by factor G .
 - Both C_{in} and C_{out} have to be multiples of G .



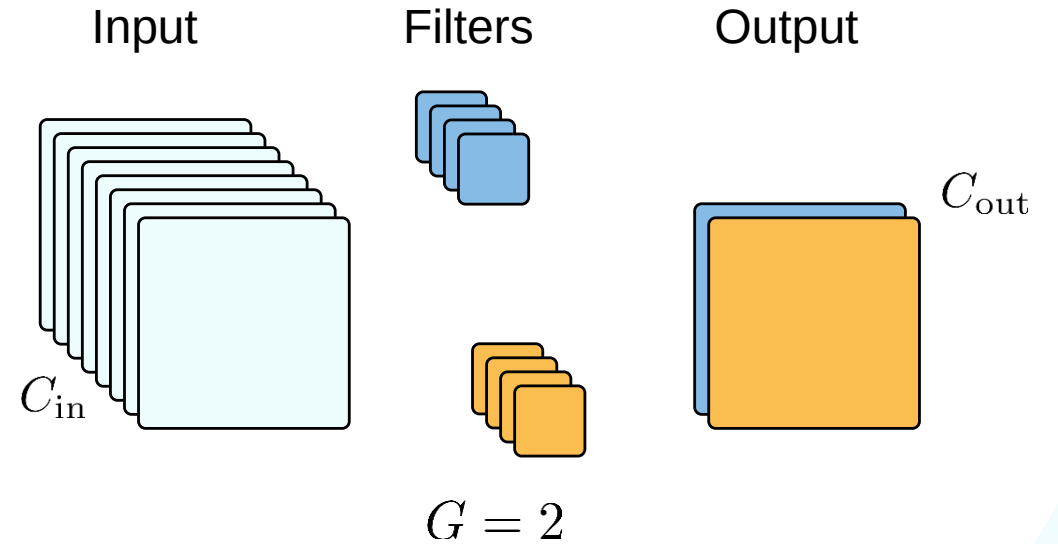
Grouped Convolution

- Divide input and output channels into G groups.
- Apply a separate convolution on each group and stack the results.
 - Reduces number of parameters by factor G .
 - Both C_{in} and C_{out} have to be multiples of G .



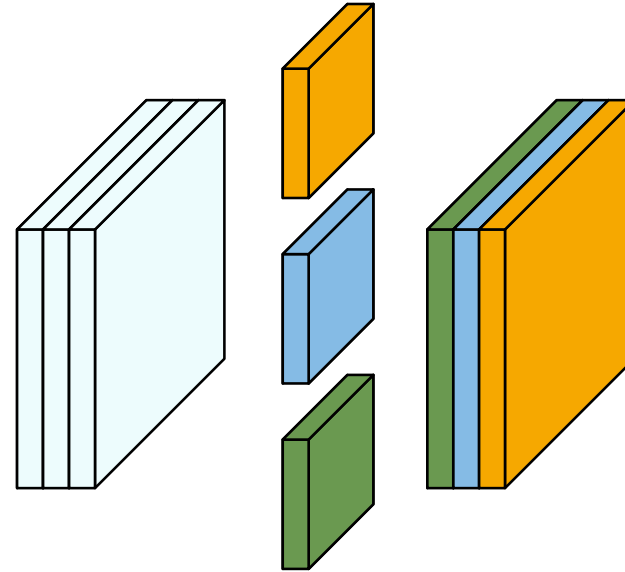
Grouped Convolution

- Divide input and output channels into G groups.
- Apply a separate convolution on each group and stack the results.
 - Reduces number of parameters by factor G .
 - Both C_{in} and C_{out} have to be multiples of G .



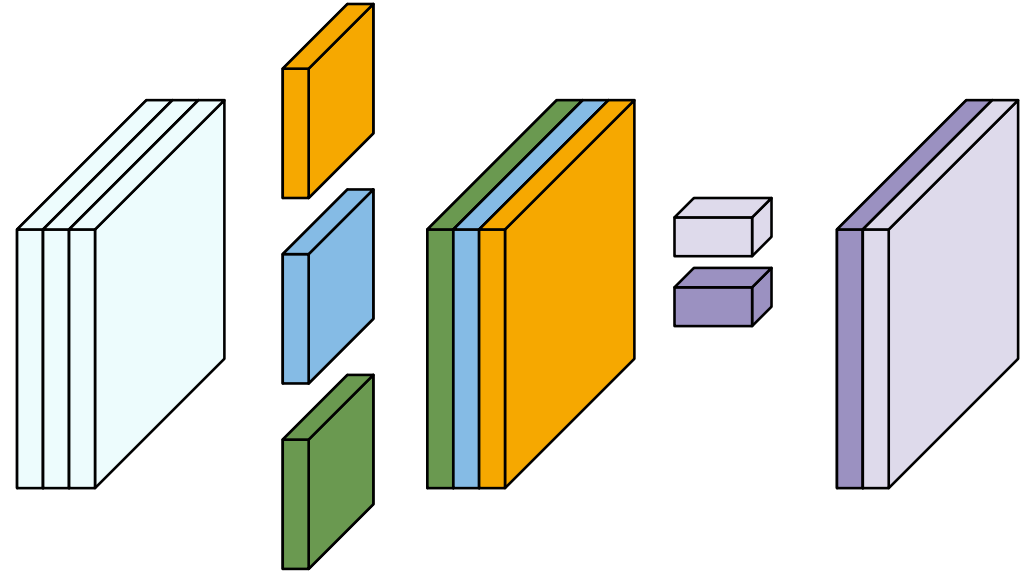
Depthwise Convolution

- When $G = C_{\text{in}} = C_{\text{out}}$, we call it **depthwise convolution**.
- Depthwise convolutions save the most parameters.
- However, channels cannot communicate with other channels.



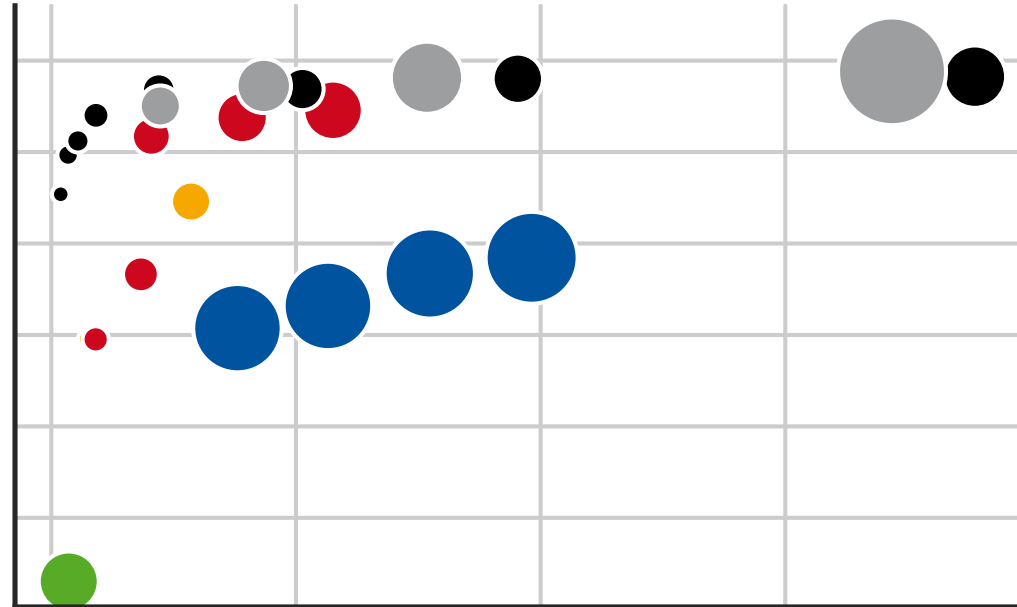
Depthwise Convolution

- When $G = C_{\text{in}} = C_{\text{out}}$, we call it **depthwise convolution**.
- Depthwise convolutions save the most parameters.
- However, channels cannot communicate with other channels.
- Solution:
 - Add a 1×1 convolution.
 - This combination is called **depthwise separable convolution**.



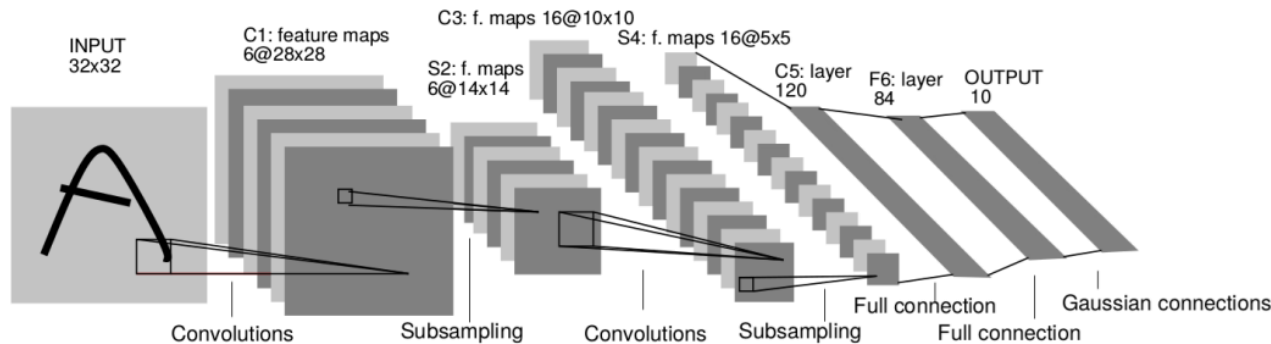
Convolutional Neural Networks

1. Motivation
2. Learnable Filters
3. Convolutional Layers
4. Other CNN Layers
5. **CNN Architectures**
6. Encoder-Decoder Structures

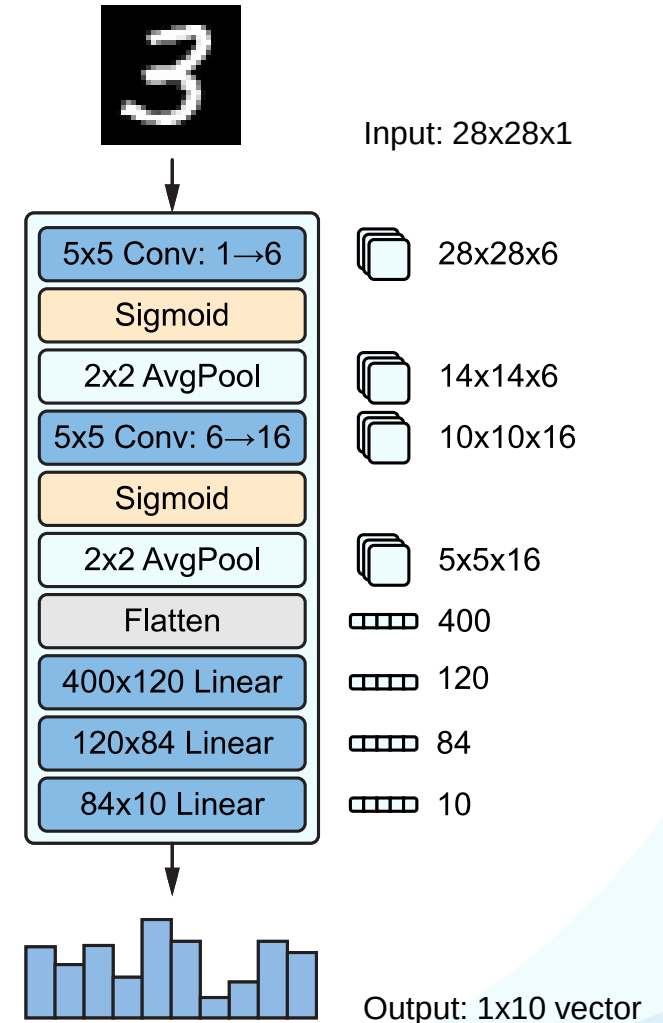


LeNet (1998)

- One of the first widely successful convolutional architectures.
 - E.g., handwritten digit recognition for postal zip codes.
- 2 convolutional layers, 2 pooling layers.
- Fully-connected NN layers for classification.
- Trained on several thousand images (low-resolution, black-and-white).



Y. LeCun et al., "Gradient-based learning applied to document recognition",
 Proceedings of the IEEE 86(11): 2278–2324, 1998.



Breakthrough Moment: ImageNet Challenge 2012

- **ImageNet**
 - Community effort to collect ~14M labeled internet images
 - 20k classes
 - Human labels via Amazon Mechanical Turk
- **ImageNet Challenge (ILSVRC)**
 - 1.2 million training images
 - 1000 classes
 - Goal: Predict ground-truth class within top-5 responses
 - One of the top benchmarks in Computer Vision



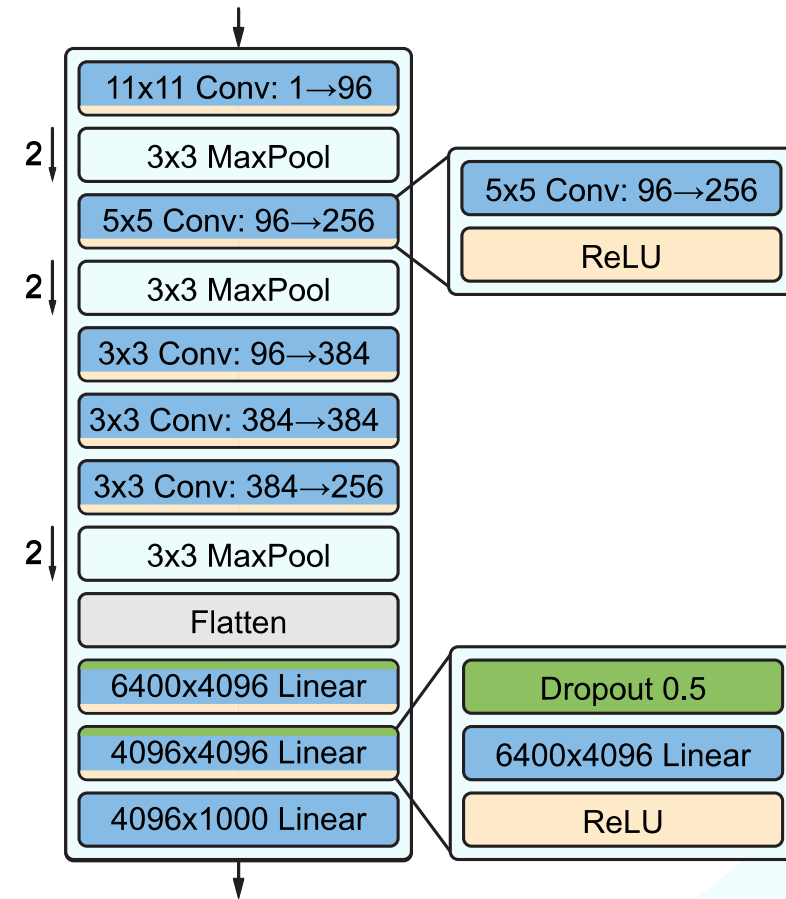
Image from [J. Deng et al., CVPR'09]

J. Deng et al., "ImageNet: A Large-Scale Hierarchical Image Database", CVPR 2009.

O. Russakovsky, J. Deng et al., "ImageNet Large Scale Visual Recognition Challenge", International Journal of Computer Vision, 115(3):211-252, 2015.

AlexNet (2012)

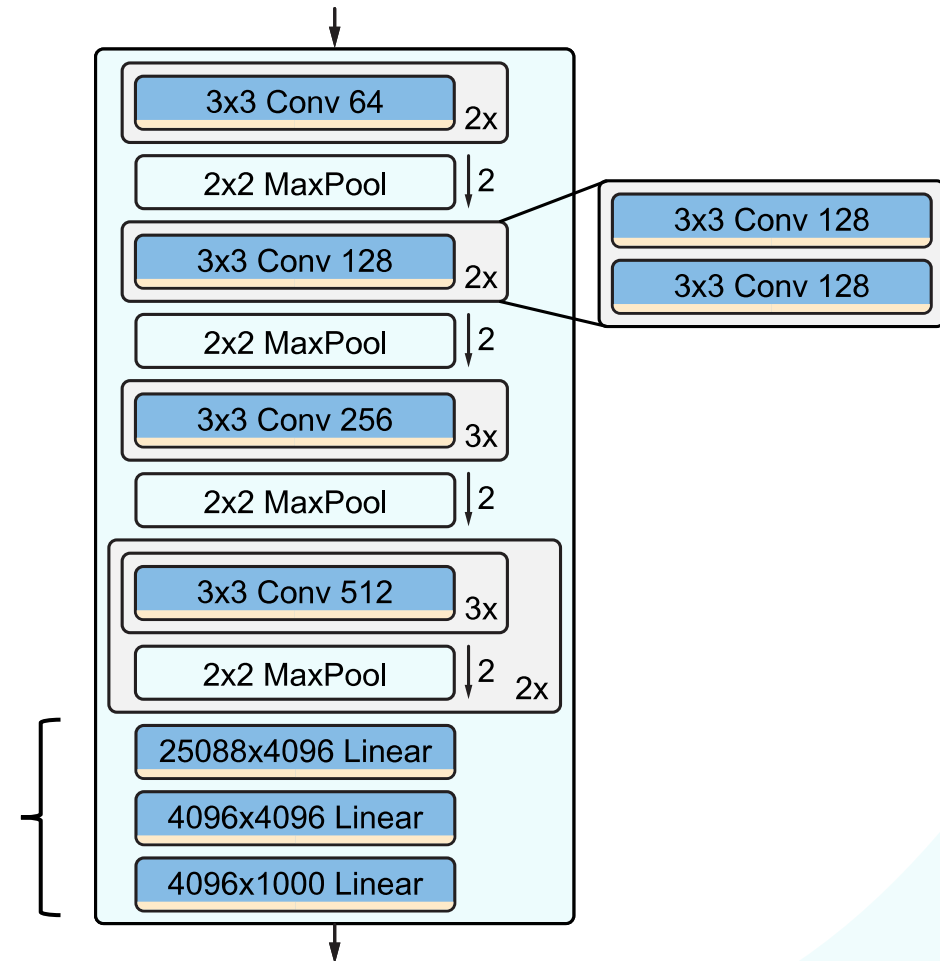
- Won the ImageNet Challenge 2012 (ILSVRC).
 - Outputs a prediction over 1000 classes!
- Bigger model than LeNet
 - 7 learnable layers, 60M parameters.
 - Used effective training strategies (Dropout, ReLU)
- Trained on about a million images.
 - Possible because of efficient GPU implementation.
- Achieved an ILSVRC top-5 error rate of 16.4%
 - Huge improvement over traditional Computer Vision methods at the time (best competitor: 26.2%)
 - *This kicked off a revolution in Computer Vision*



VGGNet (2014/15)

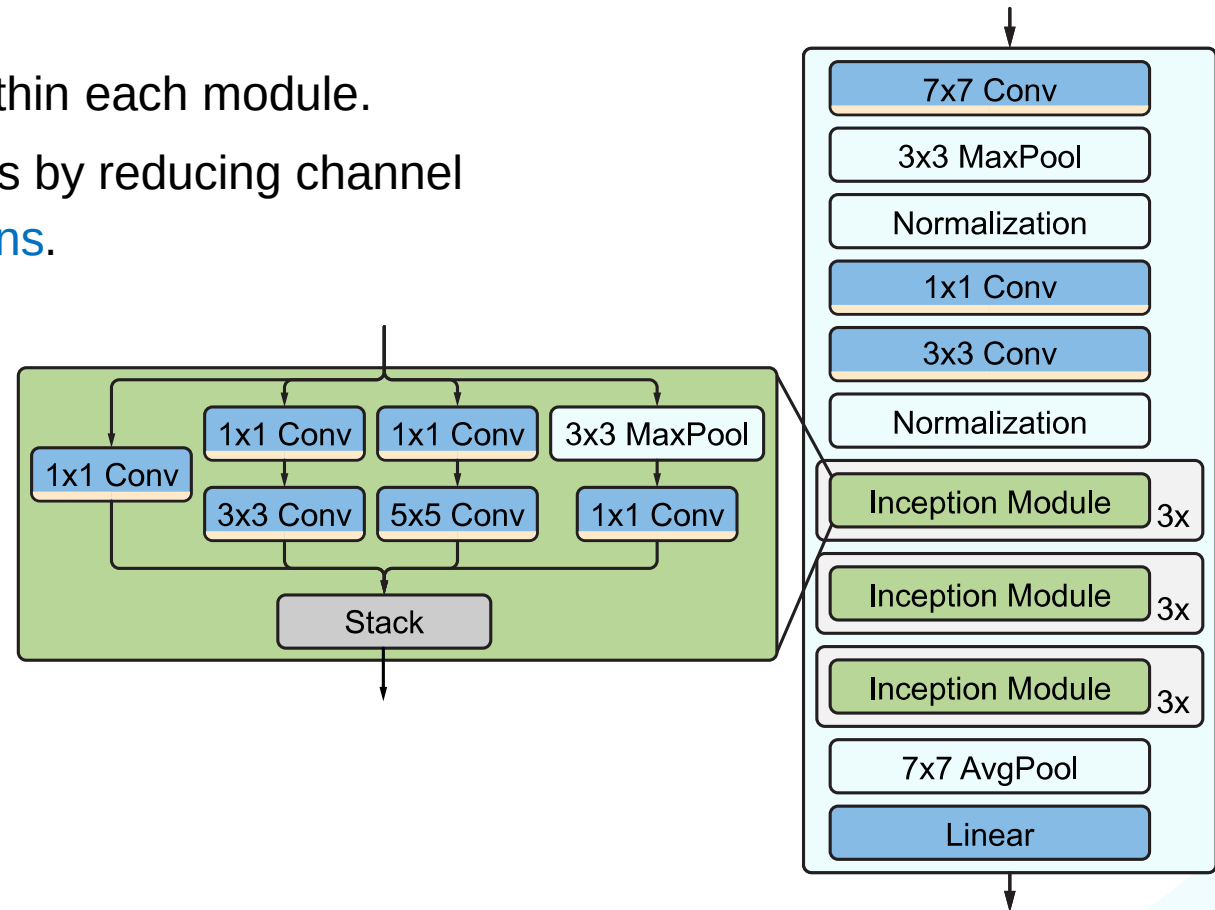
- Much deeper network than AlexNet.
 - Up to **19 layers** with 150M parameters (VGG-19)
- Stacked convolutional layers with smaller filters.
 - Stacking increases receptive field with less parameters than using a large kernel.
- Double number of channels after pooling layers (**channel inflation**).
- Improved ILSVRC top-5 error rate to 6.7%.

Most parameters are here (120M!)



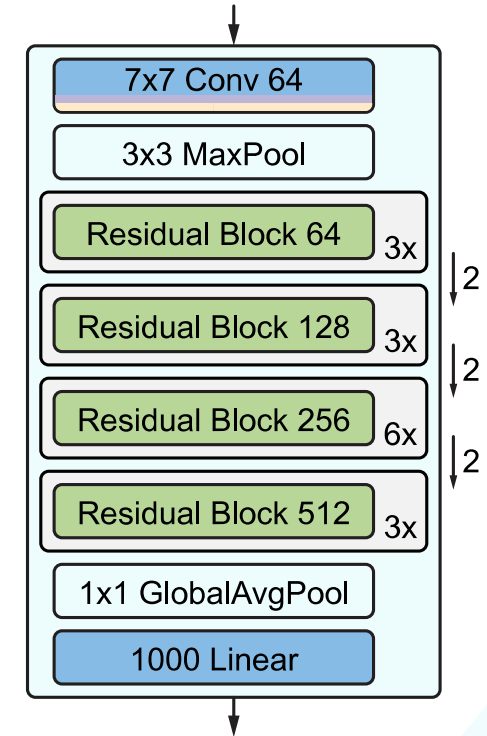
GoogLeNet (2014/2015)

- Main component: **Inception Module**.
 - Learn filters at several scales within each module.
 - Reduce complexity of large filters by reducing channel count with cheap **1x1 convolutions**.
- Replace heavy linear layers with much cheaper **average pooling**.
 - Saves *millions* of parameters compared to VGG.
 - Resulting in a network with just 6.8M parameters
 - ILVSRG top-5 error rate of 6.7%



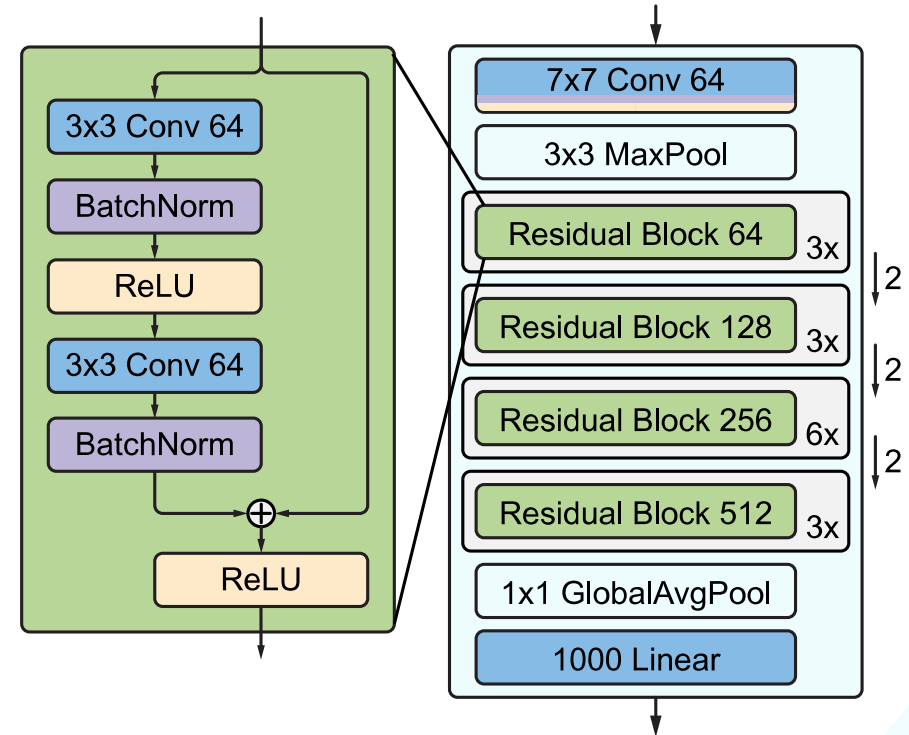
ResNet (2015)

- Family of **deep** networks (up to **100+ layers**).
 - Four stages with varying number of blocks.
 - Downsample input after first layer and after every stage.
 - Reduced ILSVRC top-5 error rate to 3.6%.



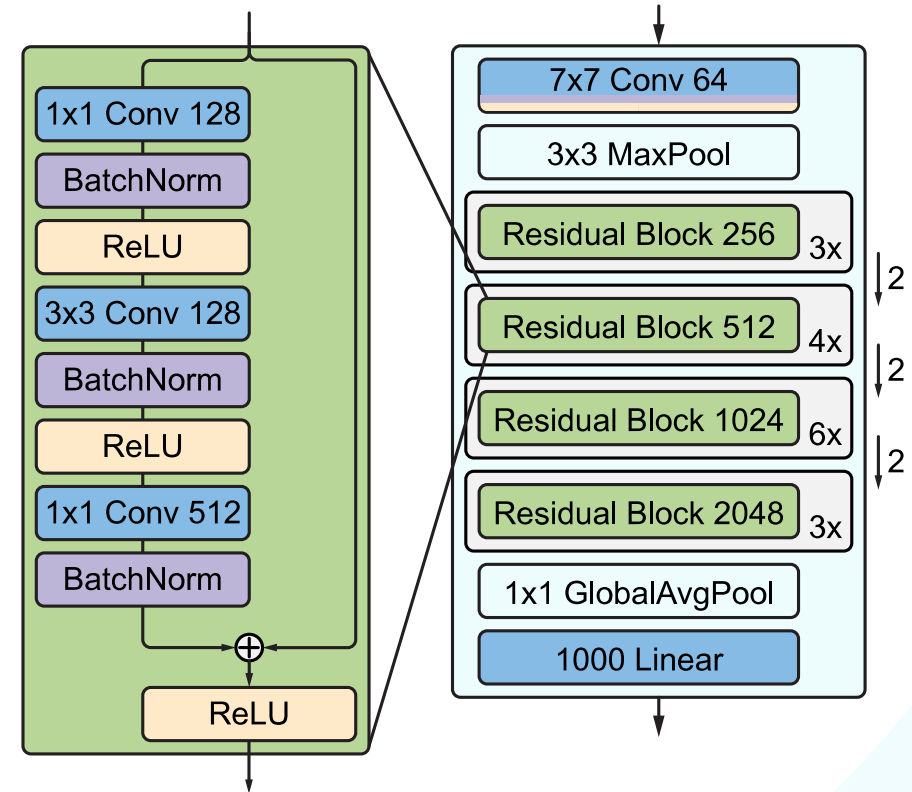
ResNet (2015)

- Family of **deep** networks (up to 100+ layers).
 - Four stages with varying number of blocks.
 - Downsample input after first layer and after every stage.
 - Reduced ILSVRC top-5 error rate to 3.6%.
- Core component: **skip connection**.
 - Improves gradient propagation.
 - Essentially computes a residual:
$$F(\mathbf{x}) = H(\mathbf{x}) + \mathbf{x}$$
 - Can be interpreted as learning an ensemble of shallower networks.

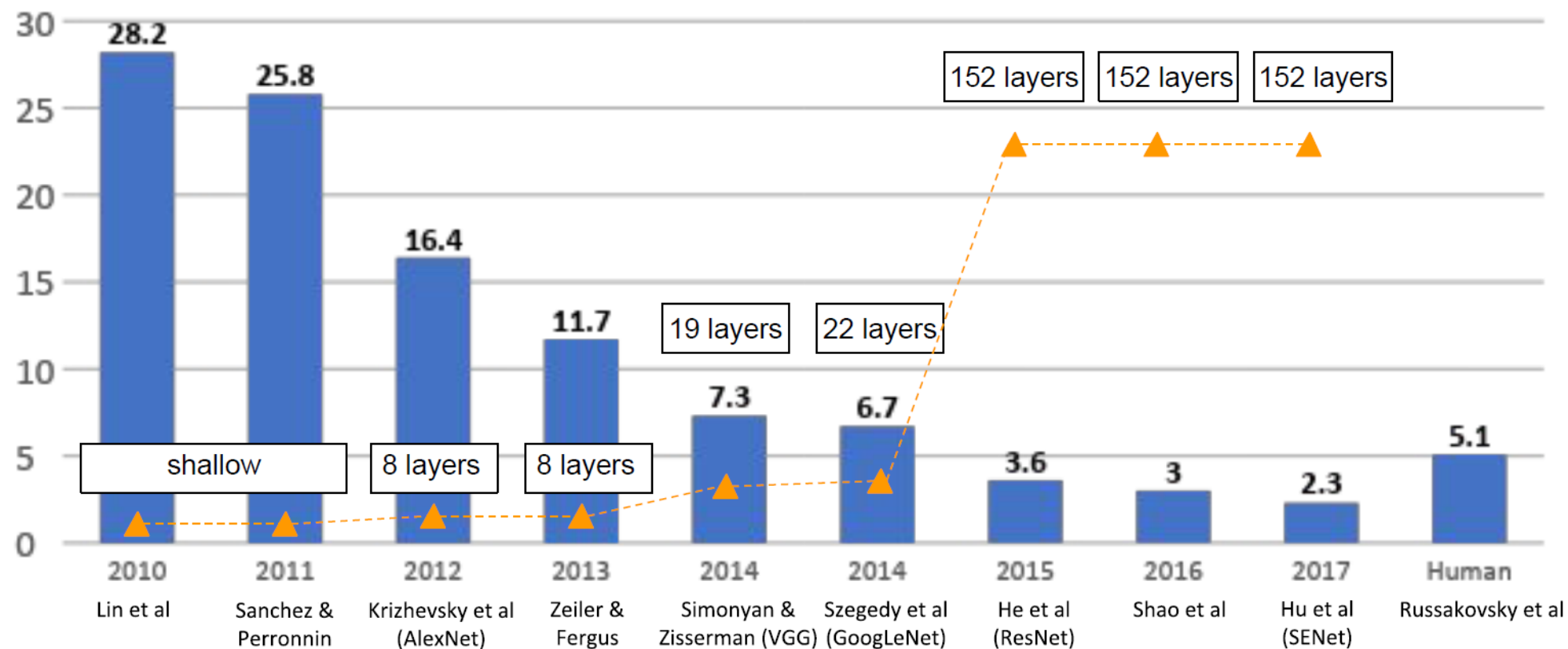


ResNet (2015)

- Deeper variants (50 layers and more) increase the number of channels a lot.
 - In each block, a 1x1 convolution projects to a lower-dimensional space to make the 3x3 conv more efficient.
- Uses **BatchNorm** by default to stabilize training.



ILSVRC Winners Over the Years



EfficientNet

- Family of networks designed for efficient training.
 - Base network found via [neural architecture search](#).
 - Scale to larger models by [compound scaling rule](#):

$$\text{Depth } d = \alpha^\phi$$

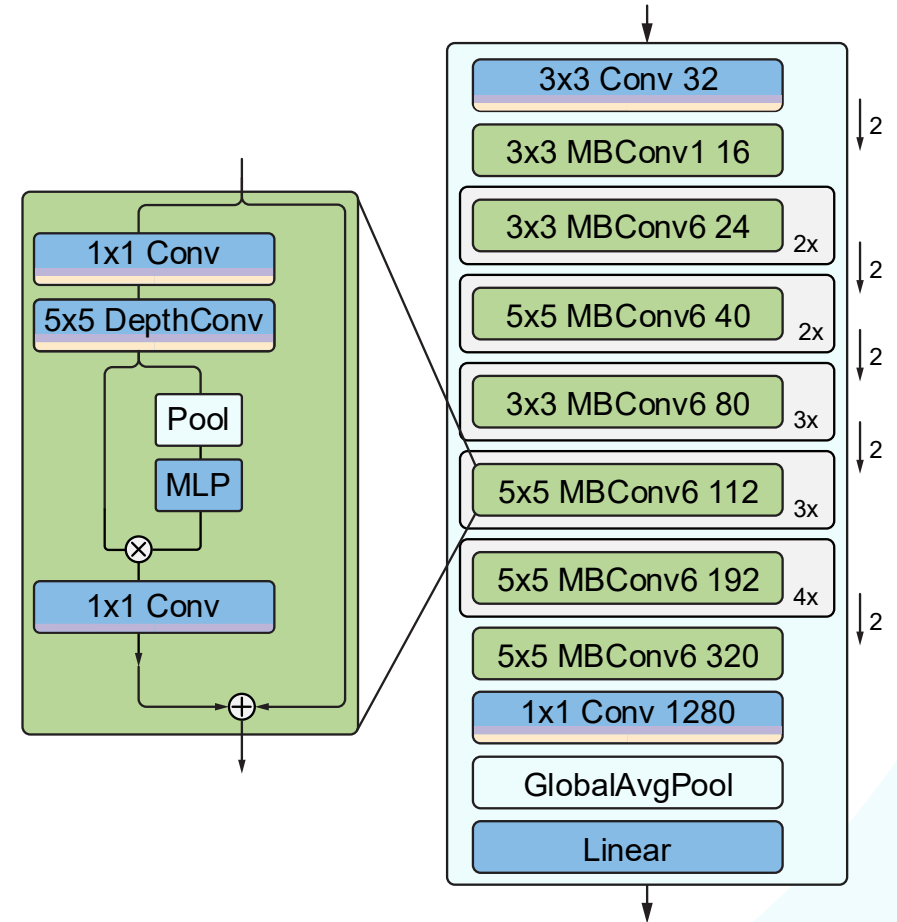
$$\text{Width } w = \beta^\phi$$

$$\text{Resolution } r = \gamma^\phi$$

$$\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$$

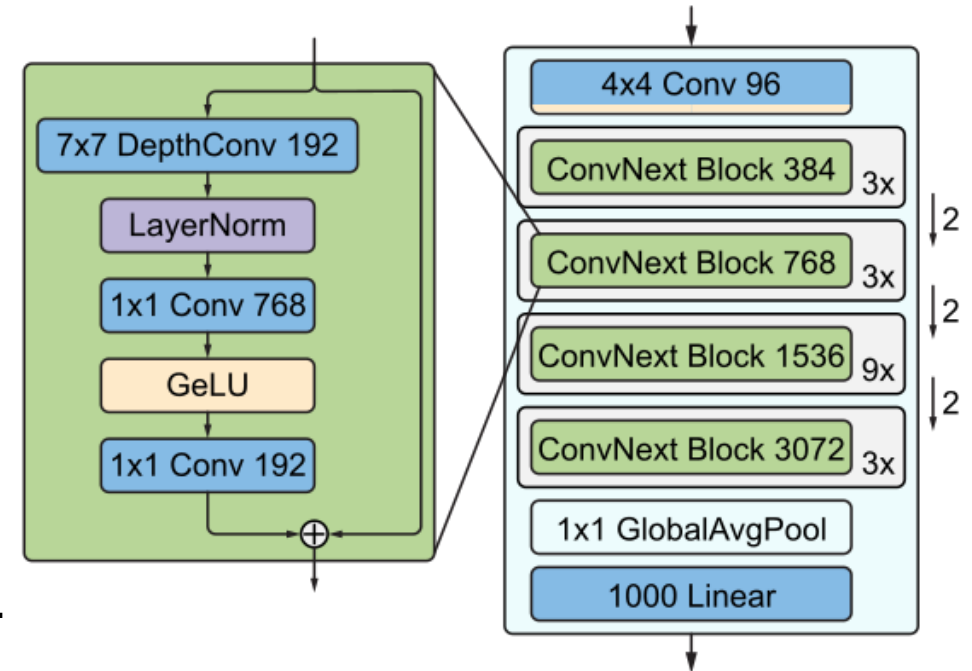
$$\alpha, \beta, \gamma \geq 1$$

- Designed for best performance with a limited compute budget.

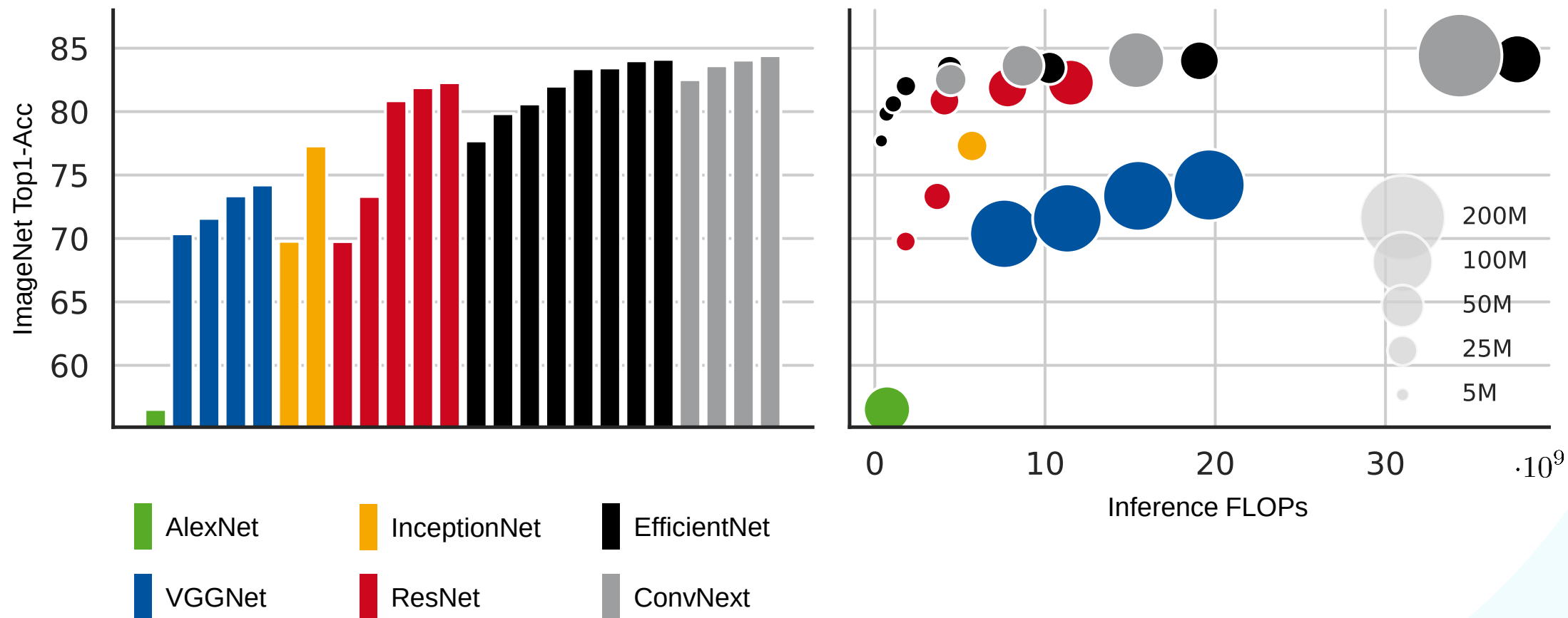


ConvNeXt (2022)

- CNN architecture inspired by transformers.
 - “Patchify” stem (linear embedding of patches).
 - Each block consists of a spatial operator and a point-wise MLP.
- Departs from established CNN building blocks.
 - **Depthwise convolution** with larger filter sizes.
 - **LayerNorm** instead of BatchNorm.
 - **GeLU** instead of ReLU.
 - Fewer activation functions + normalization layers.
 - Trained with **Adam** instead of SGD.
- Designed for maximum task performance.



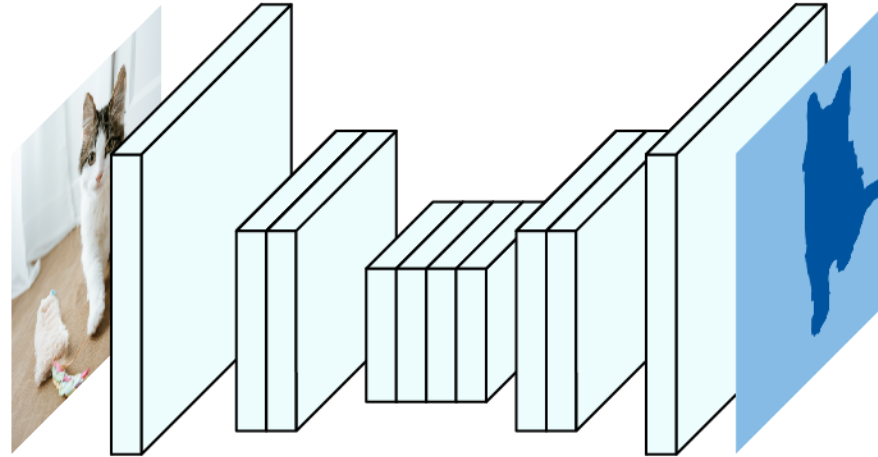
Historical Overview



Models trained on ImageNet-1K, as available in torchvision v0.17.2

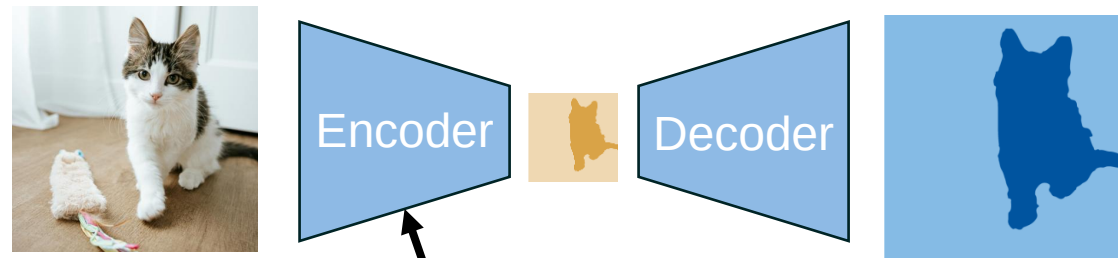
Convolutional Neural Networks

1. Motivation
2. Learnable Filters
3. Convolutional Layers
4. Other CNN Layers
5. CNN Architectures
6. **Encoder-Decoder Structures**

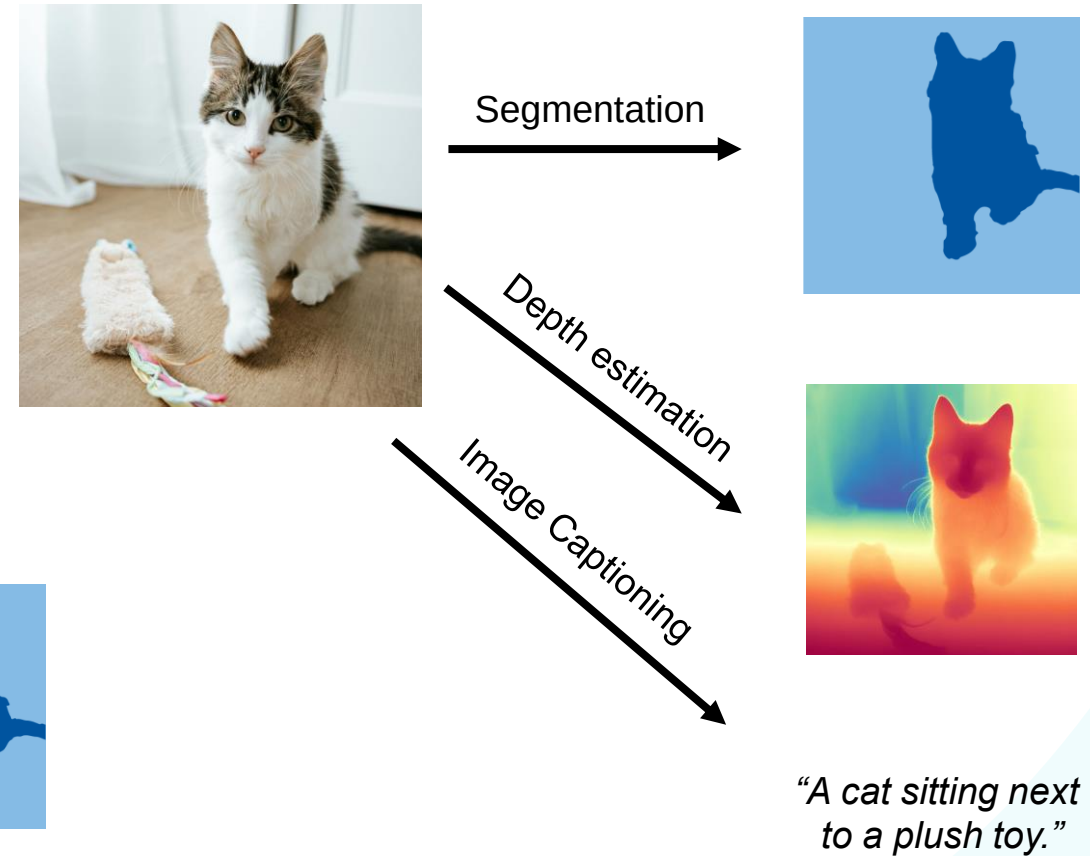


Encoder-Decoder Structures

- We can solve many advanced image tasks with CNNs.
- These models often use an Encoder-Decoder architecture:
 - **Encoder** extracts useful features.
 - **Decoder** converts them to desired output.

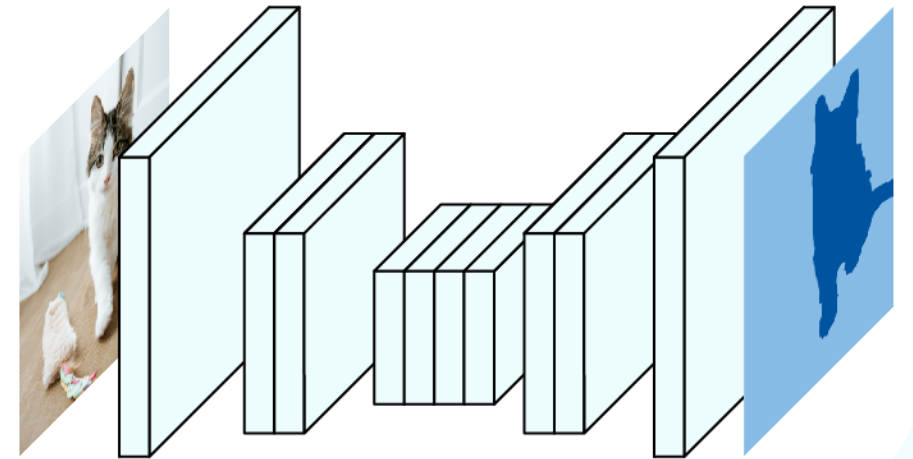


Initialize encoder with pretrained **backbone** (e.g., a classification network)!



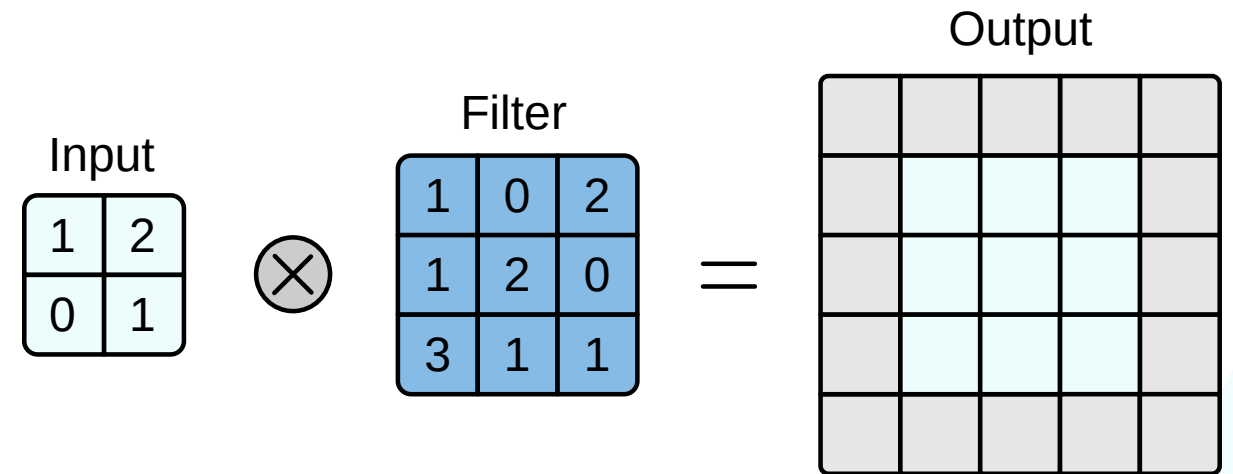
Dense Prediction Tasks

- Goal: predict label for every pixel.
- Apply loss pixel-wise.
 - E.g., pixel-wise cross-entropy for semantic segmentation.
- Problem with deep fully-convolutional networks:
 - With many channels, convolutions at original image resolution will be very expensive!
- Solution: use **downsampling** and **upsampling** inside the network.
 - Downsampling: Pooling, strided convolutions
 - Upsampling: Interpolation, **transposed convolutions**



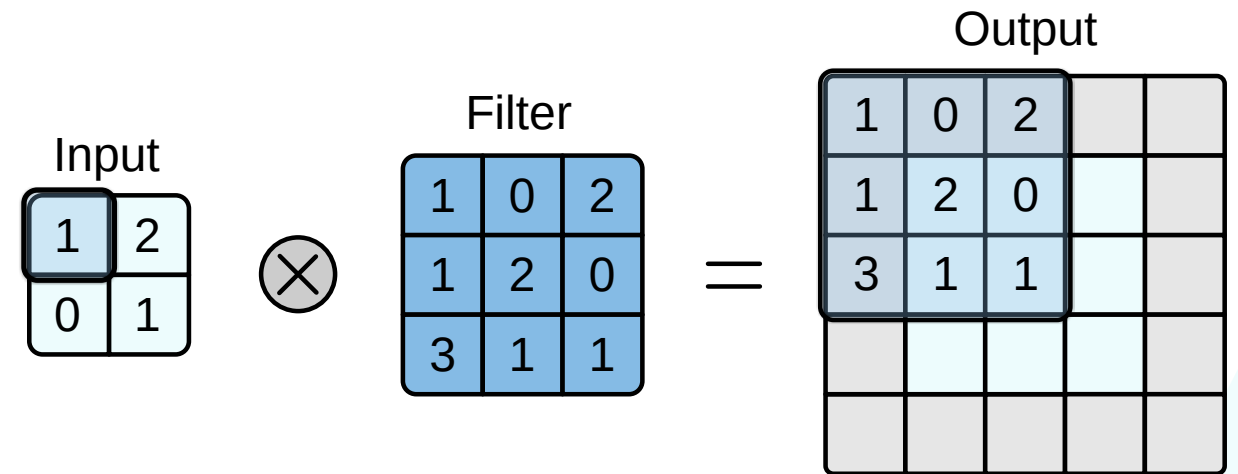
Transposed Convolutions

- For each input pixel, multiply the value with the filter and add to output.



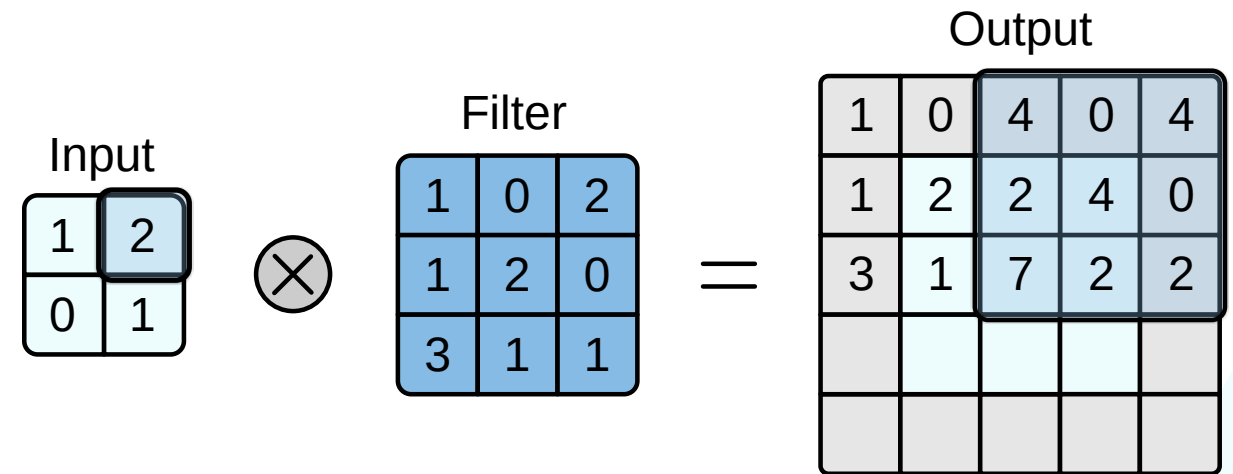
Transposed Convolutions

- For each input pixel, multiply the value with the filter and add to output.



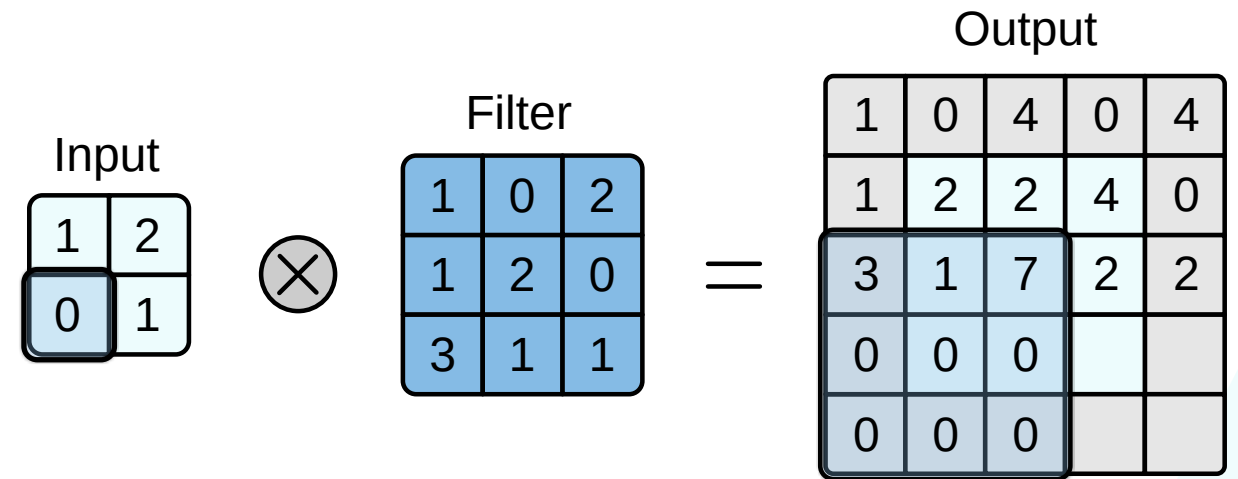
Transposed Convolutions

- For each input pixel, multiply the value with the filter and add to output.



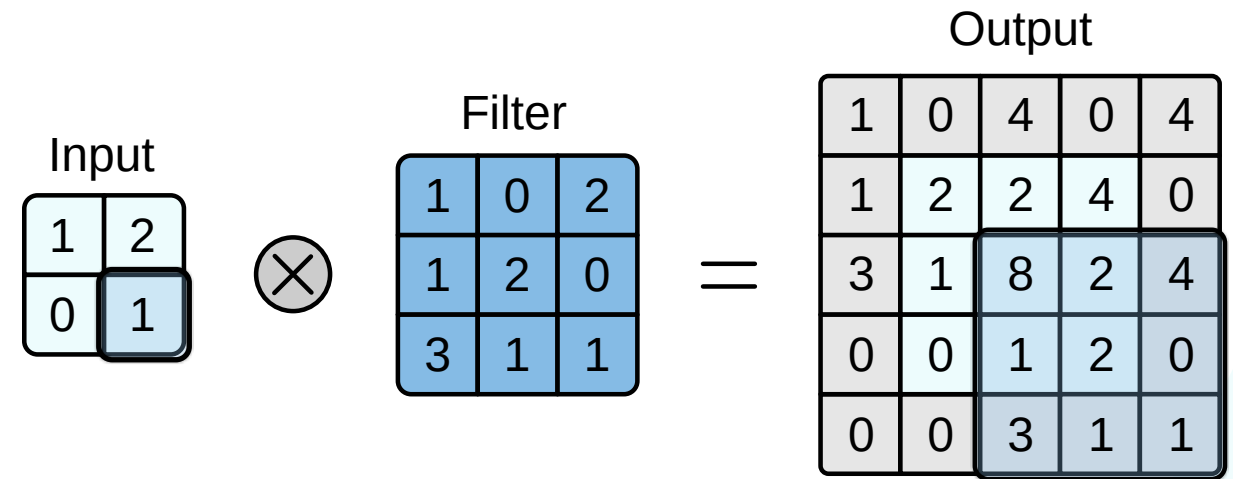
Transposed Convolutions

- For each input pixel, multiply the value with the filter and add to output.



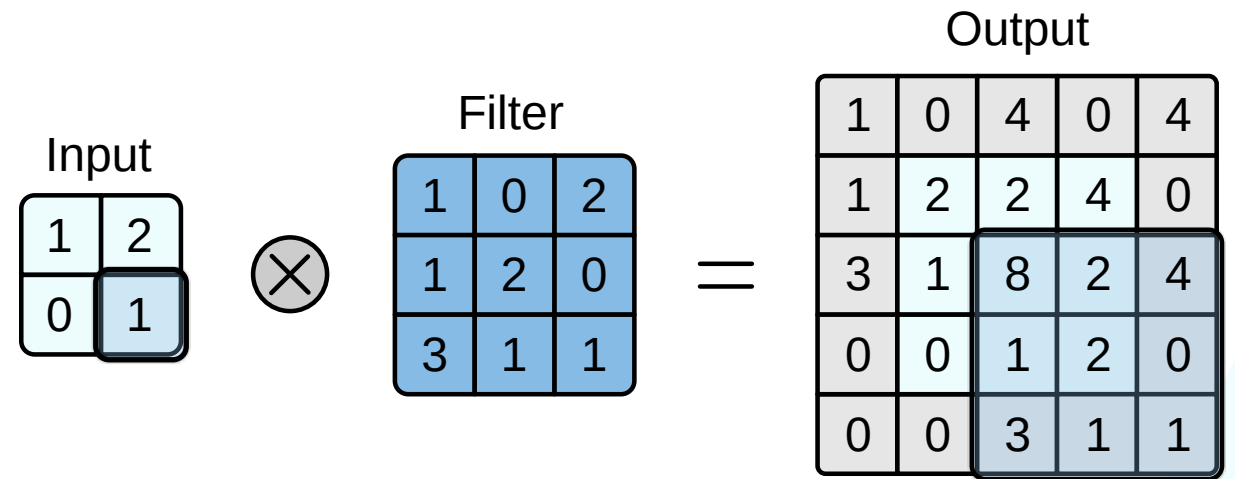
Transposed Convolutions

- For each input pixel, multiply the value with the filter and add to output.



Transposed Convolutions

- For each input pixel, multiply the value with the filter and add to output.
- Choose stride equal to filter size to avoid checkerboard artifacts.



Transposed Convolutions

- We can compute the output size as follows:

$$H_{\text{out}} = S(H_{\text{in}} - 1) + F - 2P$$

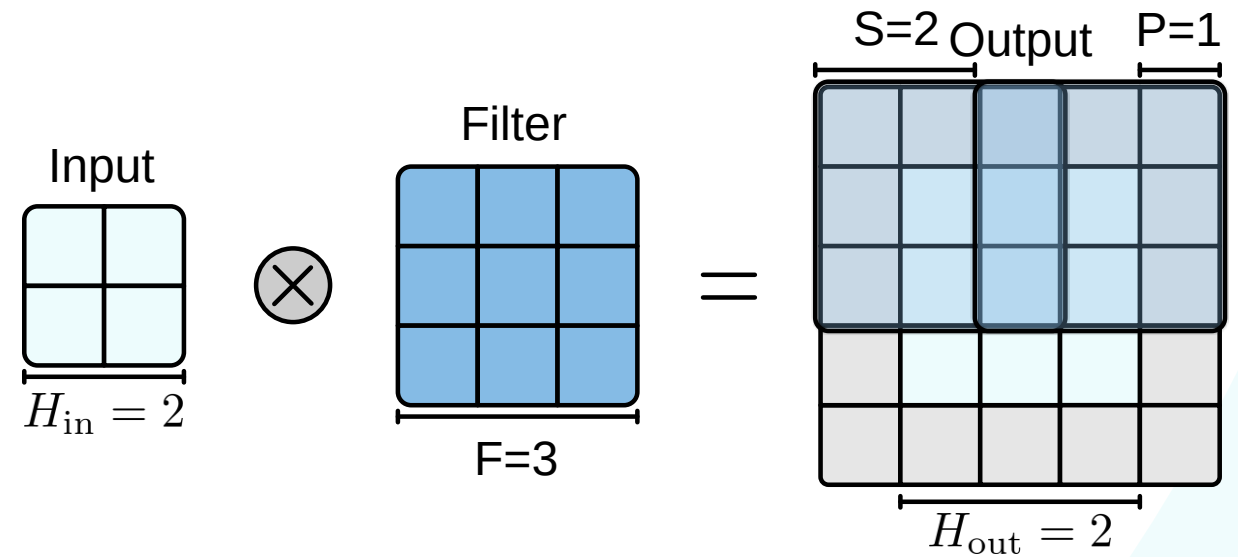
H_{in} : Input size

H_{out} : Output size

F : Filter size

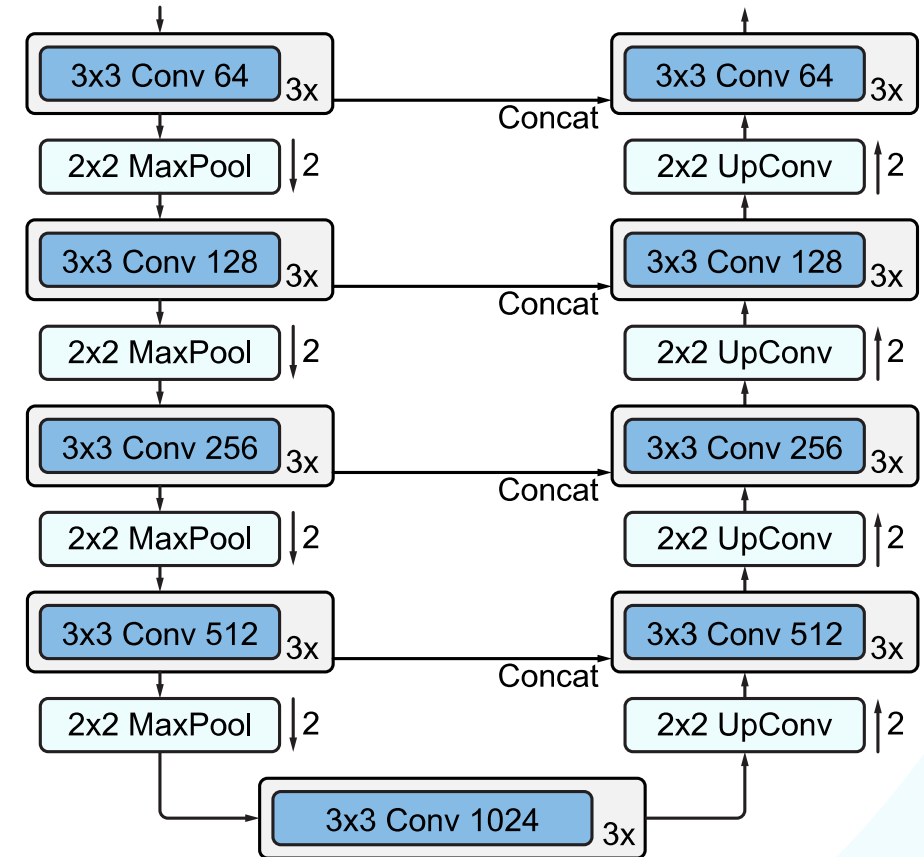
P : Padding size

S : Stride



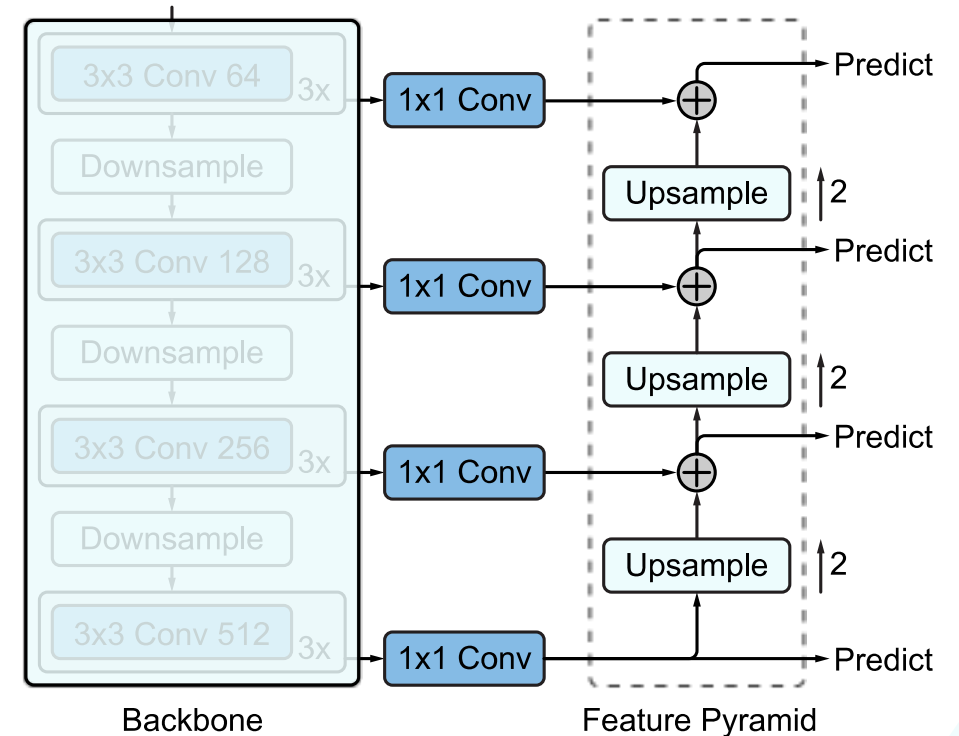
Example: U-Net

- Dense Encoder-Decoder Structure.
 - Downsample with **max pooling**.
 - Upsample with **transposed convolution**.
 - Preserve high-resolution information with **skip-connections**.



Example: Feature Pyramid Networks

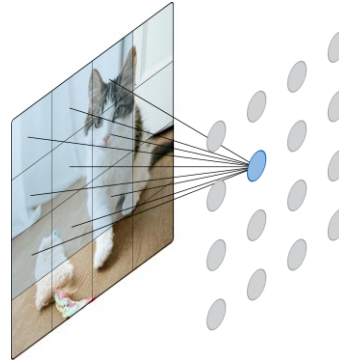
- **Feature Pyramid Networks** exploit the multi-scale hierarchy of deep convolutional networks.
 - Each level of the decoder can make a separate prediction.
- Produces a hierarchical representation of the image features at multiple scales.
 - Makes the network robust to variations in scale.
 - Provides semantic features at high resolution.
- Easy to use with different backbones.



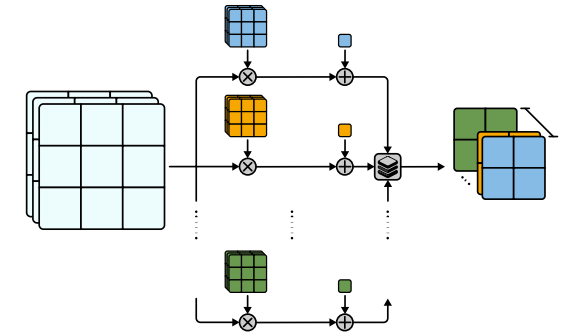
Summary: Convolutional Neural Networks



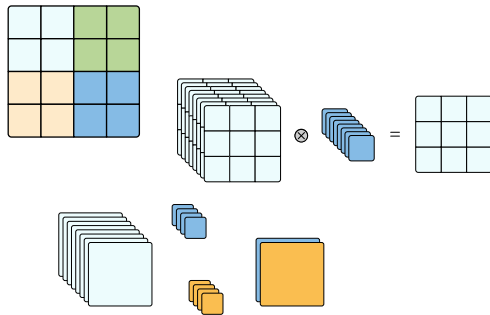
Image Processing



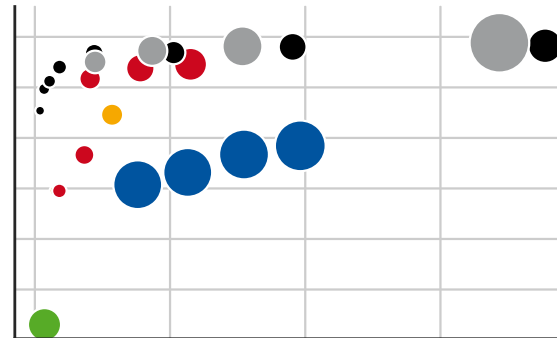
Learnable Image Filters



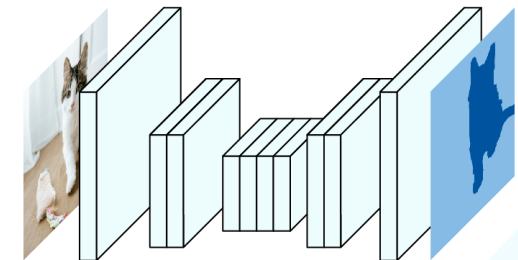
Convolutional Layers



CNN Layers



CNN Architectures



Encoder-Decoder Structures

bridging AI

Building Bridges to AI Across Disciplines

Deep Learning

Prof. Bastian Leibe

*Check out our upcoming
MOOC for more lessons
on Deep Learning!*